

LLM Context Compaction Proxy

永不限, 永不停歇 · Never Full, Never Stop

中文使用指南

LLM Context Compaction Proxy Contributors

2026 年 8 月

目录

1 功能亮点	3
1.1 上下文压缩引擎	3
1.2 记忆与会话	3
1.3 Provider 与协议	4
1.4 可靠性与安全	4
1.5 MemSkill(自演进技能,V7)	4
1.6 HTTP API 端点 (31 个)	5
2 支持的 Agent(OpenClaw / Claude Code / Hermes / DeepSeek Harness)	5
2.1 OpenClaw	6
2.2 Claude Code(Anthropic)	6
2.3 Hermes Agent	6
2.4 DeepSeek Harness	7
3 引言	7
3.1 为什么 README 是重要的	7
3.2 README 的主要组成部分	7
3.2.1 项目标题和描述	8
3.2.2 安装和使用说明	8
3.2.3 贡献指南和许可证	8
4 项目标题和描述	8
4.1 如何选择一个有吸引力的标题	8
4.2 如何写一个简洁明了的项目描述	9
4.2.1 描述的关键元素	9

5 安装和使用说明	9
5.1 提供详细的安装步骤	9
5.2 用户如何使用你的项目	10
5.3 提供帮助和支持	10
6 示例和代码片段	11
6.1 提供使用示例	11
6.1.1 Python SDK 示例	11
6.1.2 验证代理是否生效	11
7 项目结构和文件组织	12
7.1 说明项目的文件和目录结构	12
7.2 描述各个文件和目录的用途	12
7.2.1 主程序 (compaction-proxy.py)	12
7.2.2 文档目录 (docs/)	12
7.2.3 测试与配置	13
8 贡献指南	13
8.1 如何为项目做出贡献	13
8.1.1 提交代码的标准	13
8.2 提交问题和拉取请求的流程	13
9 许可证	14
9.1 选择合适的开源许可证	14
9.2 如何将许可证添加到你的项目中	14
9.2.1 许可证的类型和选择	14
10 联系信息和致谢	15
10.1 提供你的联系信息	15
10.2 致谢和表达对贡献者的感激	15
10.2.1 表达感激	15
10.2.2 社区文化	15

1 功能亮点

1.1 上下文压缩引擎

功能	说明
预压缩	上下文达 80% 阈值时提前触发, 避免触墙
溢出恢复	溢出错误时自动压缩并重试
增量压缩	只摘要自上次压缩后的新增消息 (CoMem)
并行压缩	大上下文分块并行压缩
并行摘要合并	将并行分块摘要合并为一份连贯摘要
双层压缩	网关层 (85% 缩减)+ Agent 层 (L1 的 50%), 三种降级路径
CJK 感知 Token 估算	精确统计中文/英文/其他字符 token 数 (两阶段)
压缩安全验证	压缩结果必须小于原文, 否则降级为激进截断
智能截断	基于角色的预算分配 (头尾拆分)
标识符保留	压缩时保持代码标识符完整
强制标识符列表	摘要中原样保留 UUID/哈希/URL/文件名
压缩缓存	30 分钟 TTL, 指令感知 salt 键
压缩项剪枝	清理上次压缩遗留的低价值产物
查询位置优化	查询置尾、工具对邻接、近期窗口重排
缓存优化排序	层哈希感知重排 + <code>cache_control</code> 断点
结构感知工具输出压缩	代码/日志/JSON 专用压缩器
手动压缩 (<code>/compact</code>)	类似 Claude Code 的 <code>/compact</code> , 可强制压缩任意会话
激进截断兜底	压缩失败或膨胀时降级为截断

1.2 记忆与会话

功能	说明
语义记忆	情节—语义双层记忆 (arXiv:2605.17625), 6 类知识槽位
跨会话记忆 + FTS5	SQLite 会话持久化 + 全文搜索
用户画像记忆	持久化用户偏好 (USER.md), 有界持久化
会话转录存档	保存/恢复原始转录以支持会话恢复
会话恢复	从转录 + 摘要恢复会话 (POST <code>/sessions/{id}/resume</code>)
背景知识注入	将近期会话知识注入压缩提示
会话上下文注入	自动向出站请求注入会话上下文
ARC 地址化引用	长工具结果替换为 ID 引用
ARC 持久化回查	ARC 条目持久化到数据库, 经 GET <code>/arc/{arc_id}</code> 回查
承诺提取 (CCL)	提取并保留目标/约束/决策/错误
LLM 记忆提取	LLM 驱动知识提取, 正则兜底
思考掩码	剥离推理内容以节省 token

密钥脱敏	自动脱敏 API Key/JWT/密码/Token
孤儿工具对清理	清理悬空的 tool_call/tool_result 对
工具对邻接修复	修复非邻接的工具调用/结果顺序

1.3 Provider 与协议

功能	说明
Provider 抽象层	通用 OpenAI/Anthropic/Gemini 兼容 (ABC)
自动 Provider 探测	依据模型名/请求头/URL 自动识别格式
OpenAI→Anthropic 请求转换	完整字段映射, 含 tool_calls → tool_use
Anthropic→OpenAI 响应转换	content/thinking/tool_use 映射回 OpenAI
SSE 流式转换	Anthropic SSE → OpenAI SSE 逐事件转换
Anthropic 仅思考重试	响应仅含空思考块时自动流式重试
独立压缩 Provider	压缩模型可使用独立上游与密钥
Gemini 密钥走请求头	x-goog-api-key header, 绝不进 URL/日志
模型注册表	67+ 已知上下文窗口的模型
透传路由	任意未匹配路径透明转发上游

1.4 可靠性与安全

功能	说明
熔断器	3 次失败 → 60 秒冷却, 三状态机
抖动检测	检测压缩循环 (3 次 / 5 条消息)→ 激进截断
端点认证	22+ 端点受保护; 无密钥时仅回环放行
并发安全	语义记忆线程锁、无竞态提示词 (参数传递)
压缩前后 Hooks	压缩前后的 HTTP webhook(可阻断)
健康与指标端点	/health、/metrics(Prometheus 风格计数器)
自适应保留轮次	对话过短时自动下调保留轮次
收缩窗口重试	每次重试递减保留轮次

1.5 MemSkill(自演进技能,V7)

功能	说明
技能注册表	CRUD + 激活生命周期 + 快照回滚 (持久化到 SQLite)
技能控制器	关键词匹配 + Gumbel-Top-K 选择
参数覆盖管线	每技能的重要性权重 + 保真乘数
DELETE 型技能管线跳过	跳过 DELETE 操作的语义提取

技能性能追踪	每技能奖励/成功统计 (/skills/{id}/performance)
技能轨迹	近期压缩轨迹 (/skills/trajectories)
技能设计器	自动设计新技能 (/skills/designer/trigger)

1.6 HTTP API 端点 (31 个)

方法	端点	说明
POST	/chat/completions(+/v1/chat/completions)	OpenAI 兼容对话 (流式与非流式)
POST	/v1/messages	Anthropic Messages API
POST	/compact	手动压缩会话
POST	/summarize	供 CompactionProvider 插件使用的纯摘要点
POST	/session	创建会话
GET	/sessions/search	FTS5 全文会话搜索
GET	/sessions/recent	近期会话
GET	/sessions/{id}	会话详情
GET	/sessions/{id}/transcript	原始转录导出
POST	/sessions/{id}/resume	恢复会话
GET/POST	/profile	用户画像读取/设置
GET/DELETE	/memory	语义记忆读取/清空
GET	/arc/{arc_id}	ARC 引用回查
GET	/skills、/skills/{id}	技能列表/详情
POST	/skills	创建技能 (草稿)
POST	/skills/{id}/activate、/deprecate	技能生命周期
POST	/skills/{id}/rollback	回滚到快照版本
GET	/skills/{id}/performance	技能奖励统计
GET	/skills/trajectories	压缩轨迹
POST	/skills/designer/trigger	触发技能设计器
GET	/health、/metrics	健康与指标
ANY	/path:.*	透传到上游

2 支持的 Agent(OpenClaw / Claude Code / Hermes / DeepSeek Harness)

代理已实测兼容 OpenClaw、Claude Code、Hermes 与 DeepSeek Harness 四大 Agent 框架。接入后上下文压缩完全由代理接管,Agent 自身不再压缩,可无限期运行。

2.1 OpenClaw

在 openclaw.json 中将模型 provider 指向代理, 并启用 v6-compaction-provider 插件:

```
{
  "models": {
    "providers": {
      "compaction-proxy": {
        "baseUrl": "http://127.0.0.1:8198",
        "api": "openai-completions",
        "models": [{ "id": "xopdeepseekv4flash" }]
      }
    }
  },
  "plugins": {
    "entries": {
      "v6-compaction-provider": {
        "enabled": true,
        "config": { "proxyUrl": "http://127.0.0.1:8198", "model": "
          xsparkx2agent" }
      }
    }
  }
}
```

已验证:OpenClaw 的 xunfei provider 已指向 127.0.0.1:8198, v6-compaction-provider 插件经 /summarize 走代理; OpenClaw 原生 compaction 已关闭 (compaction.enabled=false), 压缩完全由代理接管。

2.2 Claude Code(Anthropic)

```
export ANTHROPIC_BASE_URL=http://localhost:8198
claude
```

或在 7.claude/settings.json 中:

```
{
  "env": { "ANTHROPIC_BASE_URL": "http://localhost:8198" },
  "autoCompactEnabled": false
}
```

已验证:POST /v1/messages 经代理路由 (缺 key 返回 401 证明 Anthropic 格式识别成功);autoCompactEnabled: false 关闭 Claude Code 自带压缩。

2.3 Hermes Agent

编辑 7.hermes/config.yaml:

```
model:
  api_mode: chat_completions          # OpenAI 格式
  base_url: http://127.0.0.1:8198/v1  # 指向代理
  default: xopglm51                   # 上游可用模型均可
```

已验证:Hermes 的 chat_completions 模式与代理 /v1/chat/completions 完全兼容;Hermes 自带压缩已关闭 (compression.enabled: false)。

2.4 DeepSeek Harness

编辑 \$DSH_HOME/settings.yaml(默认 7.dsh/settings.yaml):

```
llm-pi-ai:
  providers:
    xfyun-maas:
      apiKeyEnv: XF_MASS_API_KEY
      api: openai-completions          # OpenAI 兼容协议
      baseUrl: http://127.0.0.1:8198  # 指向代理
      models:
        - id: xsparkx2agent
          contextWindow: 131072
          maxTokens: 8192
```

已验证:Harness 的 xfyun-maas provider 已指向 127.0.0.1:8198;Harness 原生压缩经 cordis.patch.yml 关闭 (dsh-compaction-basic → auto: false)。

3 安装和使用说明

3.1 前置条件

- Python 3.10+
- aiohttp(pip install aiohttp)

3.2 安装步骤

1. 克隆仓库

```
git clone https://github.com/061115xhsm/NeverFull-NeverStop-LLM-Context-Compaction-Proxy.git
cd NeverFull-NeverStop-LLM-Context-Compaction-Proxy
```

2. 安装依赖

```
pip install aiohttp
```

3. 配置上游并运行

```
export COMPACTION_PROXY_UPSTREAM=https://api.openai.com/v1
export COMPACTION_PROXY_MODEL=gpt-4o-mini
python3 compaction-proxy.py
```

代理监听在 localhost:8198。将 Agent 的 API 端点指向此处即可。

3.3 接入你的 LLM 提供商

提供商	配置
OpenAI / 兼容系	export COMPACTION_PROXY_UPSTREAM=https://api.openai.com/v1
Anthropic	.../api.anthropic.com + UPSTREAM_IS_ANTHROPIC=true
Gemini	.../generativelanguage.googleapis.com
DeepSeek	.../api.deepseek.com/v1
Ollama(本地)	http://localhost:11434/v1(默认)

3.4 帮助与支持

- **问题追踪:** GitHub Issues
- **健康检查:** curl http://localhost:8198/health
- **文档:** 仓库内的 README.tex 与 docs/technical-document.tex

4 示例和代码片段

4.1 Python SDK 示例

```
import openai

client = openai.OpenAI(
    base_url="http://localhost:8198/v1",
    api_key="your-key"
)

response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Hello!"}],
    stream=True
)
```

创建指向 localhost:8198/v1 的 OpenAI 客户端, 调用 chat.completions.create() 发送消息并以流式方式读取响应。

4.2 验证代理是否生效

```
# Health check
curl http://localhost:8198/health

# Send a test request through the proxy
curl -X POST http://localhost:8198/v1/chat/completions \
  -H "Content-Type: application/json" \
  -H "Authorization: Bearer YOUR_KEY" \
  -d '{"model": "gpt-4o-mini", "messages": [{"role": "user", "content": "Hello"}]}'
```

5 项目结构和文件组织

文件/目录	描述	用途
compaction-proxy.py	主程序	代理的全部核心逻辑
README.md / README.tex	项目说明	使用指南与文档
README-zh.md / README-zh.tex	中文说明	中文使用指南
docs/	文档目录	技术文档与接入指南
.env.example	配置模板	环境变量配置示例
systemd/	服务模板	systemd 用户服务单元
LICENSE	许可证	MIT 许可证文本

5.1 主程序 (compaction-proxy.py)

包含代理的全部核心逻辑:Provider 抽象层 (OpenAI/Anthropic/Gemini)、压缩引擎、语义记忆、会话存储、技能注册表、熔断器与抖动检测等。代码按模块化组织, 每个类与函数职责单一。

5.2 文档目录 (docs/)

包含技术文档与接入指南, 帮助用户和开发者理解与使用项目。

5.3 测试与配置

.env.example 提供完整配置模板,systemd/compaction-proxy.service 提供系统服务化部署模板, 方便生产环境使用。

6 贡献指南

贡献开源项目不仅包括代码, 还包括文档更新、问题报告、新功能建议等。

6.1 贡献流程

1. **了解项目**: 阅读文档和代码, 了解项目的目标、架构和设计原则;
2. **找到贡献机会**: 查看问题跟踪器, 找到可解决的问题; 关注未来计划与里程碑;
3. **贡献代码**: Fork 项目 → 本地开发测试 → 提交 Pull Request。

6.2 提交代码的标准

- 代码必须符合项目的编码标准和风格指南;
- 提交的代码必须通过所有测试;
- 代码应包含单元测试, 确保功能的正确性。

6.3 提交问题与拉取请求

方面	提交问题	提交拉取请求
标题	明确、具体	清晰、描述目的
描述	详细、包含重现步骤	详细、解释更改的必要性
附加信息	屏幕截图、动画	符合编码和风格标准

7 许可证

本项目采用 **MIT License**, 详见仓库根目录的 **LICENSE** 文件。

MIT 许可证允许他人自由使用、修改和分发你的代码, 只要他们包含原始许可证和版权声明。

7.1 许可证的类型

- **MIT License**: 宽松许可, 允许自由使用/修改/分发, 保留版权声明即可;
- **GNU GPL**: 要求使用、修改或分发该许可证下代码的人都必须将其更改公开, 并使用相同许可证。

8 联系信息和致谢

8.1 联系信息

如果你有任何问题或建议, 请随时通过 **GitHub Issue** 与我们联系, 或在项目仓库的 **Discussions** 中发起讨论。

8.2 致谢

感谢所有为本项目贡献过代码、文档、测试与建议的开发者与用户。你的每一次提交、每一个 Issue、每一条评论, 都在推动这个项目向前。

8.3 社区文化

我们欢迎每一个人的参与和贡献, 无论你的技能水平如何, 都有你的一席之地。

结语

LLM Context Compaction Proxy 是一段探索之旅的产物: 从研究论文中的压缩技术, 到生产环境中的工程实践; 从解决自己的上下文超限之痛, 到帮助更多开发者的 Agent 无限期运行。愿这份中文指南能成为你的灯塔, 指引你在上下文压缩的世界里, 找到属于自己的路径。

——*LLM Context Compaction Proxy Contributors*